

App\([Http\Controllers\Photos\AdminController](#) class, route the controller as

```
Route :: get('foo', 'Photos\AdminController@method');
You can define routes and middleware to a controller. To define a route:
Route :: get('user/{id}', 'UserController@show');
```

Now, when a request matches the specified route URI, the show method on the UserController class will be executed.

Middleware can also be assigned to controller's route or within your controller's constructor. E.g. to assign auth middleware to UserController in profile route

```
Route :: get('profile', [
    'middleware' => 'auth',
    'uses' => 'UserController@showProfile']);
```

To assign auth middleware using the middleware method in the UserController's constructor,

```
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Http\Requests;
use App\Http\Controllers\Controller;
class UserController extends Controller {
    public function __construct(){
        $this->middleware('auth');    }}
```

You can also create a single controller for a bunch of HTTP requests. This is done by creating a resource controller

```
php artisan make:controller PhotoController --resource
```

This creates the PhotoController for all requests related to photos stored on the application. This resource controller located at [app/Http/Controllers/PhotoController.php](#) will contain a method for each of the available resource operations. The generated controller will already have methods stubbed for each of these actions, including notes informing you of the HTTP verbs and URIs they handle.

Register a resourceful route to the controller:

```
Route::resource('photos', 'PhotoController');
```

Responses

Your website or application should interact with the user. So for every request the user makes, there should be a response sent to the users browser. The most basic response is returning a string from a route or controller. The